

A Vision-Based Method for Real-Time Traffic Flow Estimation on Edge Devices

Duong Nguyen-Ngoc Tran[✉], *Graduate Student Member, IEEE*, Long Hoang Pham[✉], *Member, IEEE*, Huy-Hung Nguyen[✉], and Jae Wook Jeon[✉], *Senior Member, IEEE*

Abstract—Traffic flow estimation is an essential task in modern intelligent transportation systems. Many types of information, including vehicle type, vehicle totals, and movement direction, are vital for mitigating transportation-related tasks and effective traffic control strategies. With the development of embedded devices, systems can process captured video at the edge instead of transferring data to centralized processing servers. This paper proposes a real-time and edge-based traffic flow estimation system. The proposed system follows a detect-and-track mechanism where lightweight deep learning models perform vehicle detection. A novel scenario-based tracking and counting technique is developed to provide multi-class, multi-movement vehicle counting. The method uses predefined regions to assign the movement for each vehicle initially detected. It then performs spatial-temporal trajectory matching between the vehicle trajectory and the movement path throughout the whole video. Extensive experiments have shown that the proposed method achieves high effectiveness with multiple camera types and viewpoints.

Index Terms—Intelligent transportation system, edge computing, traffic flow estimation, real-time performance.

I. INTRODUCTION

INTELLIGENT transportation systems (ITSs) [1] are crucial components in modern cities. ITSs carry out a wide range of sensing, analysis, and communication applications that process and share information to ease congestion, improve traffic management, minimize environmental impact, and increase transportation benefits. In recent years, traffic flow estimation (TFE) [2] from cameras has played a critical role in providing real-world traffic information, such as vehicle types and counts. The definition of traffic flow volume [3] indicates that counting vehicles is the most straightforward way to estimate traffic. Understanding the movement gives insight into semantic and analyzing data which can be illustrated in many forms, such as charts, graphics, images, numbers, and tables. A better basis for anomaly identification and traffic prediction may

be created by having a semantic knowledge of traffic flows in different scenarios. Furthermore, advanced TFE systems need to count vehicles that follow predefined movements from multiple camera scenes. For instance, cars need to be counted separately if they are turning left or right or moving through traffic at a given intersection. Traditionally, technicians manually count the number of vehicles. They observe a location, typically over a span of hours, and record the number of vehicles as they pass. Therefore, more significant or complicated intersections with high volume may require multiple observers. Thus, robust automated counting methods are needed because manual counting generally requires high labor costs. Subsequently, traffic flow estimation systems implemented the cloud computing architecture where all images are transferred to centralized processing servers. Problems with this approach include server overloading, frame loss and jittering, and the high costs of network bandwidth, which place a considerable burden on implementation and maintenance. Presently, with the improvement of edge-based AI, the estimation can process on the node and send the result to the server.

Several key tasks need to be considered when developing advanced vision-based TFE systems. First, vehicle detection is performed by either traditional vision methods (e.g., background subtraction [4]) or deep learning methods (e.g., SSD [5], YOLO models [6], [7]). Recent developments in deep learning-based vehicle detection have shown remarkable accuracy improvements in characterizing crowded scenes [8], [9]. Second, robust vehicle tracking from the detection results (e.g., bounding boxes) is needed to trace each vehicle's spatial-temporal trajectory. The trajectories are crucial for determining the correct counts and the movement directions. Third, vehicle movement identification in intersections is difficult because many movements are overlapped with each other (e.g., the combined, crossing, and turning lanes). Moreover, bad weather conditions and various camera viewpoints further increase the complexity of the problem.

This paper proposes a robust traffic flow estimation system operating on edge devices. The proposed TFE system follows the detect-and-track approach where vehicle detection results are used to obtain trajectories throughout the video sequence. In the system, vehicle detection is performed by lightweight deep learning models. A novel scenario-based tracking method is introduced to extract both the spatial-temporal trajectories and the movement direction of the vehicles. In this method, the movement of interest (MOI) region and a reference trajectory

Manuscript received 28 January 2022; revised 11 August 2022, 25 December 2022, and 8 March 2023; accepted 21 March 2023. Date of publication 13 April 2023; date of current version 2 August 2023. This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. 2021-0-01364, An intelligent system for 24/7 real-time traffic surveillance on edge devices). The Associate Editor for this article was D. Wu. (Corresponding author: Jae Wook Jeon.)

The authors are with the Department of Electrical and Computer Engineering, Sungkyunkwan University, Suwon 16419, South Korea (e-mail: duongtran@skku.edu; phlong@skku.edu; huyhung91@skku.edu; jwjeon@skku.edu).

Digital Object Identifier 10.1109/TITS.2023.3264796

1558-0016 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See <https://www.ieee.org/publications/rights/index.html> for more information.

are defined for each movement direction. The MOI region is a predefined polygon shape placed along the road (see Fig. 6). The reference trajectory is a set of anchor points determined by averaging several vehicles' tracks to create the average track and then simplifying it with the Ramer-Douglas-Peucker (RDP) [10] algorithm (as explained in Fig. 5). For each vehicle, the best movement direction is identified by matching the reference trajectory and anticipating the corresponding region for each movement direction. Counting is performed for each movement direction when the vehicle exits the frame. This mechanism can guarantee accurate counting, even in the presence of misdetection in one or several frames, which can be caused by the lightweight nature of the vehicle detection models. More details of the proposed TFE system can be found in Fig. 3. The vision-based TFE system is expected to be an alternative stability and scalability solution to centralized methods. Hence, the requirements for real-world processing are high, especially on embedded computing devices, such as edge-based graphics processing units. The proposed system has been implemented and tested on an NVIDIA GPU-based computer and NVIDIA-embedded computing platforms in this study, including a Jetson TX2, a Jetson Xavier NX, and a Jetson AGX Xavier [11], [12]. The experiments have shown promising performances. In summary, the main contributions of this study are as follows:

- Implemented an edge-based TFE system with high accuracy and real-time performance when monitoring complex scenes, such as an intersection with 12 lanes and an outdoor environment with heavy rain.
- Developed a novel scenario-based tracking and counting method for vehicle tracking, movement identification, and counting.
- Demonstrated the real-time processing ability of the proposed system on edge devices, such as a Jetson AGX Xavier, a Jetson Xavier NX, and a Jetson TX2.
- Showed that the results of the proposed TFE system are comparable with other state-of-the-art methods.

The rest of this paper is organized as follows. Section II discusses the related works and methods in detail. Section III describes the proposed method. Section IV shows the optimizing process and qualitative results of the proposed method. Section V presents the conclusions of this work.

II. RELATED WORKS

In this section, some relevant algorithms in traffic flow detection are reviewed. The estimation systems should provide high accuracy and consistent long-term operation to provide average daily traffic patterns for road design. These methods utilize a wide variety of concepts, so it is crucial to have a brief look at these methods to familiarize the reader with traffic flow estimation.

The first approach for traffic flow estimation is to employ a density-aware method [8], [13], [14]. This algorithm learns the feature of an object to approximate the counting results. However, it only estimates the probable number of objects instead of the exact number and cannot determine the vehicle's route. This approach tends to handle simple scenes such as

TABLE I
LIST OF ABBREVIATIONS

List of Abbreviations	
Intelligent transportation systems	ITSs
Traffic flow estimation	TFE
Movement of interest	MOI
Ramer-Douglas- Peucker	RDP
Scenario-based movement matching	SMM
Observation zone	Obz
Central processing unit	CPU
Graphics processing unit	GPU
Bounding box	bbox
Identity	ID
Kalman filters	KF
Single Object Tracking	SOT
Intersection-over-union	IOU
convolutional neural network	CNN
Weighted root mean square error	wRMSE
Normalized weighted root mean square error	nwRMSE
Frames per second	FPS
Region Trajectory	RT
Interest Region Trajectory	IRT

uni-directional traffic, but it struggles to handle two-way roads or intersection scenarios.

A detection-aware approach uses the detection and tracking results for counting vehicles or pedestrians to address the above issues. It can provide more precise estimation results. A cross line is used to record the number of vehicles passing through the mark. Reference [9] monitors a counting line at the center of the road and only studies the short-term vehicle track. However, it cannot deal with intersections and the time spent in and out of a vehicle. Reference [2] uses a virtual detection line at the end of a road, but it can only process the time spent crossing the line and it only works well on a two-way road and when there is no cross branch line. To settle a complication, [15] uses a more complex cross line and an observation zone, but it is ineffectual for cases of occlusion and missing tracked vehicles.

Other methods use the same basic zone definition framework where an intersection is divided into a single road, counting a turn based on the zones traversed. Reference [16] divides a crossroad into several non-overlapping regions. Each region defines either an adjacent road or the center of a crossing. However, this algorithm is intended only to operate on a junction where two roads converge. Reference [17] uses the same zone as [16] and illustrates a definition of a zone by regions encoded with lines. Nevertheless, this method gets confused when it comes to showing the vehicle entrance time and the time leaving the observation zone, where time is vital for implementing a traffic control strategy. Therefore, zone techniques are simple but are ineffective when vehicle tracking is lost, resulting in poor zone localization and low accuracy.

Another prominent technique is to use a predefined route for obtaining the vehicle movement. Reference [18] focuses on the relation localization points of the vehicle and route, which result in high accuracy in determining the movement and verifying the transit time of the vehicle. However, a relation can mismatch among routes. Reference [19] pays attention to

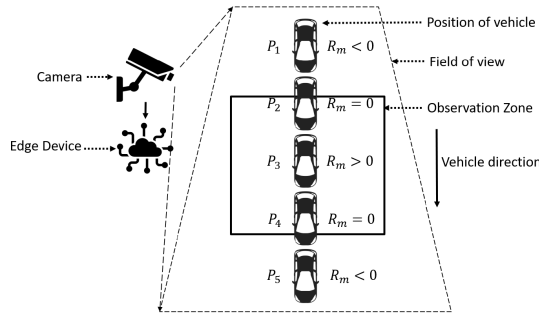


Fig. 1. Position related to the observation zone where the object is considered in the process. The dashed line trapezoid indicates the view that the camera can capture. In this area, we define the observation zone and use it to consider the state of the vehicle.

trajectory-related properties (e.g., the traveled length, direction) to store tracking information that follows the predefined trajectories and rejects small tracklets. Therefore, occluded and undetected features can be solved by maintaining the trajectory in consecutive frames.

Additionally, the running speed and deployment capability on a small device are crucial. Reference [2] prunes the object detection model to run on Nvidia Jetson devices. Reference [20] uses a small model and optimizes the tracking method to obtain higher-speed performance. Reference [21] augments and enhances the counting method to reduce unnecessary computation while maintaining high accuracy.

This paper proposes the Traffic Flow Estimation framework, which includes the use of advantageous observation points within the zone and a predefined route to obtain high accuracy while demonstrating a successful optimization process and the capability for deployment on Jetson devices.

III. METHODOLOGY

This section describes more details about scenario-based movement matching (SMM) for the TFE scheme on an edge device based on deep learning (see Fig. 7). This method has been inherited from [21] and has been further extended to improve the robustness. The scheme is primarily driven by real-time state-driven processing. We divide our system into four sections. First, in Section III-A, we describe each state of the moving vehicle, corresponding to its position in the observation zone (Obz) (see Fig. 1). We redefine the state of the object and the switching condition to help the system avoid mismatching the moving object and to give a higher accuracy flow estimation with an exact timestamp. Second, in Section III-B, we show the vehicle detection model training and deployment for each scenario. Then, we introduce all the models and weights to ensure that TensorRT runs smoothly on Jetson devices. Finally, we compensate the appearance information of object candidates by using an object-detection task with multiple object-tracking methods to compute trajectories. Third, Section III-C demonstrates scenario-based movement matching for TFE, which works in both regular routes and road intersections. Instead of manually defining reference object appearances, we use the efficacy method to characterize them. Fourth, Section III-D illustrates how we

migrated and deployed our proposed method to edge devices. The implementation is developed using a multi-threading process on the batch size input to optimally utilize the full capability of a central processing unit (CPU) and graphics processing unit (GPU) (see Fig. 7).

A. Moving Object State

When moving vehicles travel through the camera's field of view, state management supervises, defines, and adjusts their existence in the whole program by giving them states based on their positions (see Fig. 1). The state of a moving vehicle is represented by

$$S_{i,m} = f(A_{i,m}, R_{i,m,Obz}, S_{i-1,m}) \quad (1)$$

where $R_{m,Obz}$ is the relation between the position of the moving object m and the region of interest (ROI). The relation is calculated by

$$R_{m,Obz} = \alpha D_{m,Obz} \quad (2)$$

where the coefficient $\alpha = \begin{cases} -1 & \text{at } P_1, P_5 \\ 0 & \text{at } P_2, P_4 \\ 1 & \text{at } P_3 \end{cases}$ corresponding to a position of the object with respect to the Obz; $D_{m,Obz}$ is the distance from the center of the vehicle to the nearest edge of the Obz with

$$D_{m,Obz} = \min_{e_{Obz} \in E_{Obz}} D_{m,e_{Obz}} \geq 0 \quad (3)$$

and e_{Obz} is the edge of Obz with a set of edges E_{Obz} . $A_{i,m}$ is the age of the object, which is determined by a bounding box (bbox) matching condition:

$$A_{i,m} = \begin{cases} A_{i-1,m} + 1 & \text{if } \text{bbox match} \\ A_{i-1,m} & \text{otherwise} \end{cases} \quad (4)$$

Later, the age of the object is used as the condition of classifying whether the object is tracked and whether it will become a moving vehicle.

There are seven vehicle states in the system:

- **Outside:** The vehicle is detected but not considered a moving object ($S_m = 0$). The term refers to a vehicle whose position is outside of the Obz.
- **Candidate:** The new moving object is created and waits to meet the tracking requirement to become a new track ($S_m = 1$). The term is the initial state of any detected vehicle.
- **Confirm:** The moving object is considered a moving vehicle with its own trajectory ($S_m = 2$). After the object with the Candidate state matches the tracking requirement, the system assigns the Confirm state to it.
- **To be counted:** The moving vehicle reaches the end of the Obz and is considered for matching ($S_m = 3$). The system changes the object with a Confirm state into the To Be Counted state if it touches the edge of the Obz, and the system prepares to calculate the flow of the object.
- **Counting:** The moving vehicle waits for trajectory matching with the appropriate movement identity (ID) ($S_m = 4$). The object whose state is To Be Counted would

change to the Counting state while the system estimates the object's flow.

- **Counted:** The trajectory of the moving vehicle has been matched, recorded, and updated while the vehicle is not entirely out of the Obz ($S_m = 5$). After finishing the flow gauging, the system adjusts the object state from Counting to Counted.
- **Exit:** The moving vehicle has just exited the Obz or frame and is marked to be deleted ($S_m = 6$). The system tracks the object whose state is Counted until it moves out of the Obz, then modifies it to an Exit state and prepares to remove it from the tracking process.

Details of the switching between each state are illustrated in (5). Each condition in the algorithm contains processes that are described in Section III-B and Section III-C.

$$S_{i,m} = f(A_{i,m}, R_{i,m}, Obz, S_{i-1,m})$$

$$= \begin{cases} 0 & \text{if } A_{i,m} > A_{thr} \quad R_{i,m}, Obz < 0 \quad S_{i-1,m} = 1 \\ 1 & \text{Initial value} \\ 2 & \text{if } A_{i,m} > A_{thr} \quad R_{i,m}, Obz \geq 0 \quad S_{i-1,m} = 1 \\ 3 & \text{if } R_{i,m}, Obz \leq 0 \quad S_{i-1,m} = 2 \\ 4 & \text{if } R_{i,m}, Obz < 0 \quad S_{i-1,m} = 3 \\ 5 & \text{if } R_{i,m}, Obz < 0 \quad S_{i-1,m} = 4 \\ 6 & \text{if } R_{i,m}, Obz < 0 \quad S_{i-1,m} = 5 \end{cases} \quad (5)$$

B. Vehicle Detection and Tracking

1) *Vehicle Detection:* For vehicle detection, the challenge is that the model must run on an edge device at an acceptable speed. One solution is that the model must be small and consume as little memory as possible. A second solution is that we can use a particular library for high-performance deep learning inference for specific edge devices, such as TensorRT. By using both solutions, we can deliver low latency and high throughput for deep learning inference applications.

We use miniature versions of pre-trained models on COCO and fine-tune them on manually annotated, specific data, which is shown in more detail in Section IV-C. However, there is an issue when using small detection models. These versions cannot handle different object instances, such as objects varying in color, texture, material, shape, and/or size. Therefore, we train each edge device with its specific data and own weight, and the detailed data is collected wherever the edge device is located. For example, in Fig. 2-d, the camera is a dome with a wide field of view. For this camera, the size of the vehicle is different compared to a bullet camera with a long-distance view (see Fig. 2-c). Additionally, cameras with the same viewpoints and vehicle scales are grouped and trained together, allowing the small models to extract good detection results. For example, the camera in Fig. 2-e and the camera in Fig. 2-f can be considered to have the same viewpoints. If we constructed an extended set from the data then fine-tuned a unified model for all devices, the sample size must be huge for the model to adapt and run well. Furthermore, a unified model with a small size finds it hard to handle the variety of appearances of vehicles.

Finally, to optimize the power of a GPU in each device and avoid bottleneck issues, we input the model with a batch size

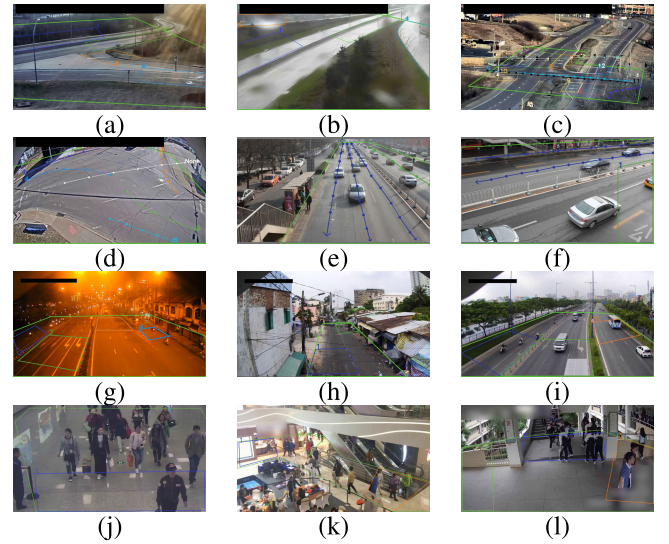


Fig. 2. Visualization of scenario-based movement matching with the observation zone (green polygon) of cameras of AI City [22], UA-DETRAC [23], TSS in Ho Chi Minh city [24], and Human in Events [25]. Each data set corresponds to each row. The colored trajectories (e.g., blue, orange, yellow ...) represent each reference trajectory or the movement of interest region, and the white trajectory represents the uncounted movement.

of images. Then, the detection results are put into a buffer queue for the tracking task. To conclude, for more detail, Section IV-C further describes the lightweight model training, and Section IV-D describes the implementation and migration on Jetson devices.

2) *Online Multi-Vehicle Tracking:* The tracking module either associates currently-tracked vehicles with newly-detected vehicles or allocates a new tracking. The vehicle enters the observation zone and is detected after several frames. Then, a new track with a unique ID is assigned for this object. The tracked locations of the vehicle in consecutive frames are used to define its trajectory. The trajectory is later used to associate it with the movement of the vehicle. The module maintains the unique ID of the vehicle while it is crossing the field of view of the camera. The tracking method also must meet the requirements for running on an edge device (i.e., real-time tracking and a small size).

The SORT [30], DeepSORT [31], and FairMOT [32] methods are used for online multi-vehicle trackers based on the tracking-by-detection paradigm. The SORT method uses efficiency algorithms, like Kalman filters, for motion prediction and the Hungarian algorithm [33] for the matching components to achieve real-time processing. This method associates already detected objects across different frames based on the coordinates of the detected bounding boxes. DeepSORT also uses Kalman filters (KF) [34] for motion prediction, but it uses deep features with a wide residual network to link detection to tracking. FairMOT uses position-aware feature maps to acquire the object with feature tracking. DeepSORT and FairMOT are slower than SORT, but they still fulfill the real-time tracking requirement. We describe the details of matching detection with a moving object in Section III-B.3. After running the tracker, we obtain the vehicle's tracklet:

$$\mathcal{T}_m = \{p_m^{t_1}, p_m^{t_2}, \dots, p_m^{t_n}\} \quad (6)$$

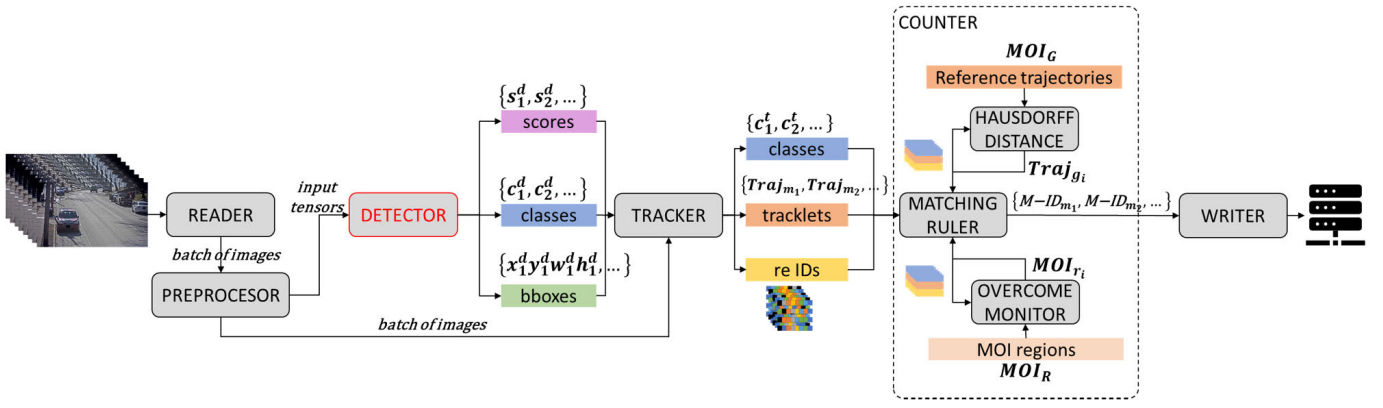


Fig. 3. The method contains six solution components, including a reader, preprocessor, detector, tracker, counter, and writer. The reader receives a traffic video stream containing multiple vehicles of interest as the input frames, concatenates the image into a batch, and sends it to the preprocessor. The preprocessor compresses and formats the frames from memory into the batch and pushes them into the memory to further process information. The detector in the GPU reads the input tensor from memory and efficiently performs batch recognition of the vehicles within view. The outcome of the detector is pushed into the memory. The tracker runs a batch of vehicle detection results from the detector to match the existing trajectory or create a new moving vehicle. The counter performs the scenario-based movement matching scheme for tracking vehicles (tracklets) in a temporally sequential manner. The writer uses the counter result to create the estimation flow and sends the outcome to the database.

TABLE II

SINGLE OBJECT TRACKING METHOD IN TRAFFIC FLOW ESTIMATION

Method	Number tracking object			
	1	5	10	20
SAOT [26]	7.733	6.187	2.320	0.232
ATOM [27]	14.667	11.733	4.400	0.440
FASNT [28]	7.325	5.860	2.198	0.220
AEPT [29]	20.800	16.640	6.240	0.624

Note: experiment on a Jetson AGX Xavier

Batch size = 1, queue size=10,

number of nodes = 1, interval frame = 1, YOLOv5s.

where $p_m^{t_i} = (x_m^{t_i}, y_m^{t_i})$ is the coordinate point of the vehicle, t_1 is the time when the vehicle is in the Candidate state, t_n is the time when the vehicle is in the Counted state, and the condition between two update tracks is:

$$t_{i+1} - t_i \geq 1 \quad (7)$$

In the case of adding Single Object Tracking (SOT) in TFE, we have to implement the manager of an object because we have to use the detector to acquire the vehicle instead of manually providing the first object for SOT. Moreover, we must execute the task of distinguishing the current tracking object from the new object provided by the detector, which makes the framework run slower than using the Multiple Object tracking. As seen in Table II, the time process for tracking is directly proportional to the number of vehicles on the road. From only one object to many objects, there is a dramatic drop in the system's speed.

3) *Data Association Scheme*: The sequential matching process associates existing tracklets to currently observed object detections. The Hungarian Matching algorithm is used to process the optimal assignment of detections to tracklet predictions in terms of intersection-over-union (IOU) cost, Mahalanobis distance cost, and Cosine distances cost, respectively, for unmatched ones. Process matching metrics by order not only can balance between the computational loads and actual input sizes for each metric, but each metric also functions in

each state condition in detection, including slow-moving, fast-running, and occluded states.

a) *IOU matching*: The cost criteria by using the IOU metric for optimal matching is from the SORT [30] approach. The metric has soundly shown that it is fast and effective when used alongside the Kalman Filter. It exploits the examination that detections of an object in consecutive frames have an unmistakably high overlap IOU, which is commonly the case when using sufficiently high frame rates or if an object is moving slowly. The IOU cost of detection D_i with any tracklet T_j defined with threshold $\epsilon_{IOU} = 0.2$ is

$$\text{cost}_{IOU}(i, j) = \frac{b_i \cap b_j}{b_i \cup b_j} \quad (8)$$

b) *Centroid matching*: In scenarios with an object rushing through the field of view or when the camera position suddenly changes, such as when mounted to an unstable drone, using IOU metrics fails to provide accurate estimates. For solving such problems, we use the Mahalanobis distance metric to adapt the algorithm to the matching of fast objects. The Mahalanobis distance, also known as statistical distance, is a distance, when given its components, takes the variability of each variable into account when determining its distance from the center. Therefore, an object with high variability receives less weight than components with low variability. In the case of detection, the Mahalanobis distance determines the proximity of the centroid in the detection D_i from the distribution of the Kalman Filter-state of a tracklet T_j . The distance express that the D_i is further away from the current bounding box, which is the mean μ of the Kalman Filter-state along its principal component axis of bounding box motion. To conclude, the cost calculated by Mahalanobis distance is defined by

$$\text{cost}_{centroid}(i, j) = (D_i - \mu_j)^T S_j^{-1} (D_i - \mu_j) \quad (9)$$

where S_j is the tracklet, T_j the covariance matrix, and μ_j is the mean of the projection of the j^{th} tracklet distribution.

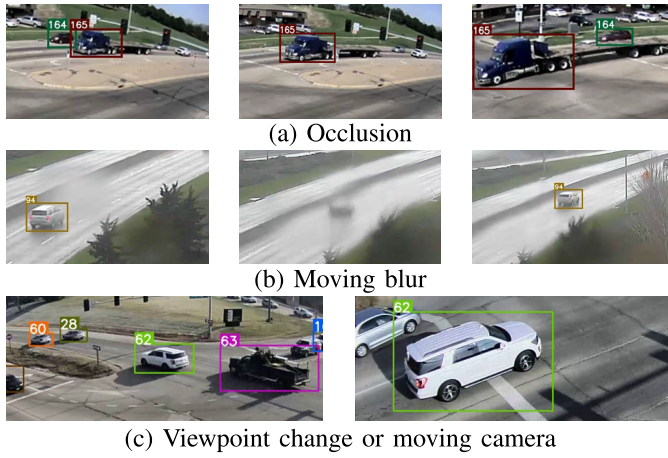


Fig. 4. The issues in a Traffic Surveillance System while keeping the identification of the vehicle. (a) The occlusion of one car covering another car. (b) The weather affects the camera. (c) The change in position of the camera or a moving camera.

c) Feature matching: In the event of occlusion, where an object is covered by others, the trajectory of the object is corrupted and the detection is missing. The result in that scenario is that the object motion is nonlinear, which cannot be effectively handled by centroid distance matching. The convolutional neural network (CNN) architecture is applied to solve this problem. Its architecture includes a wide residual network with two convolutional layers followed by six residual blocks. The network extracts the global feature map of dimensionality 128 for each object in the dense layer of 10 nodes. The final batch and l_2 normalization projects a feature onto the unit hypersphere to be fitted with the cosine appearance metric

$$\cos\theta = \frac{\sum_{i=1}^n (A_i \times B_i)}{\sqrt{\sum_{i=1}^n (A_i)^2} \times \sqrt{\sum_{i=1}^n (B_i)^2}} = \frac{A^T \cdot B}{\|A\| \times \|B\|} \quad (10)$$

where A, B are the feature map of dimensionality.

Additionally, data association is essential in tracking traffic flow for intelligent city systems to retrieve vehicles that emerge in various surveillance conditions. Re-ID [35] is one of the data association methods for matching in tracking. It belongs to the Feature Matching process we mentioned earlier in this paper. Re-ID features efficiently work under occlusion and viewpoint changes. For feature extraction, we train the Re-ID model to extract the feature from the Synthetic Vehicle X dataset [36]. In the tracking process, we obtain the feature of the cropped bounding box from detection. We average the features of all bounding boxes in each trajectory, with an area of more than 2000 pixels. In the tracker, we must tune the internal parameter for each scenario or camera to obtain higher accuracy. We set the trajectory to persist longer than 3 seconds (which means if the video has 10 FPS, we set the time before it was removed at frame #30). Moreover, we configure the vehicle so that it maintains its Candidate stage in 5 or 10 adjacent frames until it is considered as a moving object to be tracked and is assigned the Confirm state. Examples of challenges facing the tracker are shown in Fig. 4. The first issue is that of occlusion (as shown in Fig. 4-a), which happens when the appearance of one vehicle covers another vehicle, which results in the tracking missing the object for several

frames before tracking is resumed. The moment the covered car emerges from behind the truck, we can match it again with the existing trajectory based on the feature. The second issue is the hazardous weather. As shown in Fig. 4-b, we meet this problem because of the camera's water blob, which blurs the part of the image when the vehicle passes behind it. To solve the problem, the Kalman Filter still predicts the vehicle's trajectory, and the vehicle's live time is long enough to match its own trajectory by Feature Matching. The third issue is that of a change in the camera position. In this case, the other data associations cannot help, but feature matching can. As mentioned above, we keep the average feature of all trajectories, so, after appearing in another camera or the same camera with a changing viewpoint, we still keep the Identity of the vehicle by matching without creating a new tracklet.

C. Scenario-Based Movement Matching

After obtaining the series of tracklet results from the detection and tracking process, the tracklet must be matched with a predefined reference movement. This section describes a scenario-based movement matching method used to measure the number of vehicles in each movement direction. The proposed method consists of two separate vehicle movement identification methods: trajectory movement matching and the matching of the movement of interest (MOI) region trajectory matching, with the details and differences explained below.

1) Trajectory Movement Matching: We define reference trajectories corresponding to several movement directions for each camera by running reference trajectory selection. The movement direction is the orientation that vehicles follow in an intersection or road. First, following the examples in [22], the reference trajectories are modeled by accumulating and averaging several moving vehicles' trajectories from the camera. Second, the number of anchor points in the reference trajectories is subsequently reduced until the tracking robustness can no longer be maintained. Using reference trajectories with fewer points can decrease the processing time while maintaining the matching robustness (see Fig. 5). Finally, we have a completed reference trajectory set:

$$\mathcal{T}_G = \{\mathcal{T}_{g_1}, \mathcal{T}_{g_2}, \dots, \mathcal{T}_{g_l}\} \quad (11)$$

where trajectory $\mathcal{T}_{g_i}$ stands for i^{th} defined movements. Each reference trajectory has a set of points that are used to match a vehicle's tracklet:

$$\mathcal{T}_g = \{p_g^1, p_g^2, \dots, p_g^k\} \quad (12)$$

In addition, further tweaks are performed for some specific cases to further reduce the processing time. In most cases, the reference trajectories are kept from the entrance to the exit for each predefined movement (see Fig. 2-a). In some specific cases, the roads do not interact with each other; when this happens, we keep only the portion at the end of each reference trajectory (see Fig. 2-b). When fewer anchor points are used, the processing speed is faster. Moreover, in some cameras, empty $\mathcal{T}_{None}$ reference trajectories (as shown in Fig. 2-e, and Fig. 2-f with white trajectories) are defined in the set to remove unwanted vehicle movements.

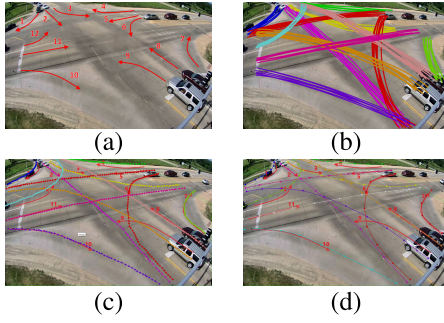


Fig. 5. Schematic diagram of reference trajectory selection modeling. (a) The defined movement, (b) the collection of satisfying trajectories after filtering, (c) the average trajectory, and (d) using the Douglas-Peucker algorithm to reduce the number of anchor points.

a) Reference trajectory selection: We proposed the use of reference trajectory selection to find the best reference trajectory for each defined movement (see Fig. 5). Reference trajectory selection can be divided into three steps: trajectory collection (see Fig. 5-b), defining the average trajectory (see Fig. 5-c) and trajectory aggregation (see Fig. 5-d).

First, for the trajectory collection step, we collect the vehicle's trajectories from each movement by considering the entirety and filtering out any distracting trajectories. The integrity of the trajectory is defined based on the start and end of the trajectory of the vehicle. With the condition of the first and last point of the trajectory falling near the edge of the Obz, and the entire trajectory going through the Obz, this trajectory is considered to have high integrity. After being collected, the trajectories are drawn with colored lines. For example, as shown in Fig. 5-b, we have different colored lines, and each color represents each defined movement in the crossroads. For more detail, the trajectory collection is described as

$$\mathcal{T}_{M_i} = \mathcal{T}_{M_i, m_1} \cup \mathcal{T}_{M_i, m_2} \cup \dots \cup \mathcal{T}_{M_i, m_n} \quad (13)$$

where

$$\mathcal{T}_{M_i, m_j} = \{p_{M_i, m_j}^{t_1}, p_{M_i, m_j}^{t_2}, \dots, p_{M_i, m_j}^{t_n}\} \quad (14)$$

is one of the trajectories that fulfills the requirement of integrity. That is, the movement m_j enters the ROI at the t_1 frame leaves the ROI at the frame t_n and the target position at time t_i is $(x_m^{t_i}, y_m^{t_i})$.

Second, the next step is to compute the average trajectory step. Each movement has several trajectories after collection, and we can average these to find the best trajectory that represents all of them. We use polynomial regression with a second-order polynomial to find the estimated curve in all anchor points. Then, any anchor point having a distance less than or equal to σ is considered in the average trajectory:

$$\mathcal{T}_{m_a} = \{p_{m_a} | p_{m_a} \in \mathcal{T}_M\} \quad (15)$$

Finally, the last step is the trajectory aggregation step. We reduce the number of anchor points while maintaining the quality by using the Ramer-Douglas-Peucker [10] algorithm. The set $\mathcal{T}_g$ of vertices p'_k divides the set $\mathcal{T}_{m_a}$ into ordered

subsets S_k of the following form:

$$S_k = \{p_i, p_{i+1}, \dots, p_j\}; p_i = p'_{k-1}, p_j = p'_k \quad (16)$$

where p_i, p_{i+1} are consecutive vertices in the curve $\mathcal{T}_{m_a}$, $i = 0, 1, \dots, N$ and p'_{k-1}, p'_k are consecutive vertices in the curve $\mathcal{T}_g$, $i = 0, 1, \dots, N$. S_k has a cardinal number that satisfies a specified criterion function $f(S_k) \leq \alpha$. Thus, we have:

$$f(S_k) = \max \left(d \left(p_i, \overline{p'_{k-1}, p'_k} \right) \right) \leq \alpha \quad (17)$$

where $p_i \in S_k$ and $\overline{p'_{k-1}, p'_k}$ represent the straight-line segment from p'_{k-1} to p'_k .

b) Matching with reference trajectory: Given the tracklets of movement in each route, each of them has the trajectory $\mathcal{T}_m$, and we use the Hausdorff distance [37] to find the best match between the tracklet $\mathcal{T}_m$ and the reference trajectory in $\mathcal{T}_g$. We use the Euclidean distance for each point in both sets. Let $d_H(m, g_i)$ denote the Hausdorff distance between the tracklet of vehicle m and reference trajectory g_i . The distance between p_m and p_g is defined as the Euclidean distance:

$$d_E(p_m, p_g) = |p_m - p_g| \quad (18)$$

The distance between point $p_{\mathcal{T}_m}$ and reference trajectory $\mathcal{T}_g$ is:

$$d(p_m, \mathcal{T}_g) = \min_{p_g \in \mathcal{T}_g} d_E(p_m, p_g) \quad (19)$$

The distance from the tracklet $\mathcal{T}_m$ to the reference trajectory $\mathcal{T}_g$ is:

$$d_H(\mathcal{T}_m, \mathcal{T}_g) = \min_{p_m \in \mathcal{T}_m} d(p_m, \mathcal{T}_g) \quad (20)$$

However, the Hausdorff distance considers two sets of points as disordered sets. Therefore, $\mathcal{T}_m$ is divided into ordered subsets, from t_n to t_1 , to choose the best match (even if points were lost during tracking), where $sub_m \subseteq \mathcal{T}_m$. Based on (18), (19), (20), we show the distance from the subset of the tracklet to the reference trajectory:

$$d_H(sub_m, g) = \max_{p_{sub_m} \in sub_m} \min_{p_g \in \mathcal{T}_g} d_E(p_{sub_m}, p_g) \quad (21)$$

Each camera has its distance threshold D_{thr} . If the distance value satisfies the condition:

$$d_H(sub_m, g) \leq D_{thr} \quad (22)$$

where sub_m matches the reference trajectory $\mathcal{T}_g$. The reference matches with greater sub_m of the tracklet are assigned as the MOIs of a moving object.

2) Movement of Interest Region Trajectory Matching: In addition to the reference trajectories, we provide several movement of interest (MOI) regions on the road, and each movement direction has one MOI. The MOI region's primary purpose is to capture tracklets that are too short (i.e., the trajectory length is too short) to match any reference trajectories. Short tracklets can result from misdetection, fast-moving vehicles, and new vehicles breaking out from occlusion.

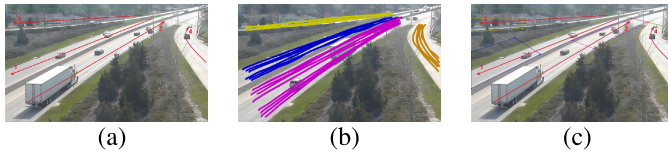


Fig. 6. Schematic diagram of the movement of interest region trajectory matching modeling. (a) The defined movement, (b) the collection of satisfying trajectories after filtering, and (c) the scenario where the MOI is based on the roadside.

a) *Interest region trajectory selection*: We propose the use of reference trajectory selection to find the best ROI for the defined movement (see Fig. 6). Interest region trajectory selection can be divided into two steps: trajectory collection (see Fig. 6-b) and MOI aggregation (see Fig. 6-c). For trajectory collection, we use the same steps used for trajectory movement matching, as described above. For MOI aggregation, the region is based on the roadside and cover each collection's last part.

b) *Matching with interest region trajectory*: Whenever a vehicle has been moving through the MOI region, it is assigned a corresponding movement ID. We have a list of MOI regions:

$$\mathcal{M}_R = \{\mathcal{M}_{r_1}, \mathcal{M}_{r_2}, \dots, \mathcal{M}_{r_l}\} \quad (23)$$

where

$$\mathcal{M}_r = \{p_r^1, p_r^2, \dots, p_r^h\} \quad (24)$$

When the point of vehicle trajectory p_m is in the polygon $\mathcal{T}_r$, the ROI is assigned to the MOI of moving object.

3) *Matching Method Combination*: Trajectory movement matching and movement of interest region matching are used in vehicle movement identification (see Fig. 3). Each vehicle movement ID is shelved until the vehicle reaches the end of the Obz or goes out of the image frame. Finally, the vehicle movement ID and the exiting time are recorded and output to the database. By measuring the number of vehicles based on their movement, we can estimate the traffic flow in sequences of images taken at intersections.

D. Implementation of the Pipeline

The main pipeline of the implementation is shown in Fig. 7. The pipeline utilizes a buffer queue and multithread processing to diminish the bottleneck between the detection and tracking stages. Specifically, detection is performed in a separate thread, and the detected bounding boxes are buffered in a queue. The detected bounding boxes are clustered together by the frame ID to maintain temporal consistency. The tracker, running on another thread, gets the detection results and performs its task as soon as the buffer queue is available. We improve the system performance by leveraging current hardware developments for edge devices and propose a thread-level framework for optimizing work on each device.

We implement an object detection model on the GPU, with a tensor format input from the temporary memory. We deploy five threads of continuous processing on the CPU and initialize all necessary components of the solution, including a reader, preprocessor, tracker, counter, and writer. Details about each of these threads are illustrated in Fig. 7 and described below:

1) *Thread 1 - Reader*: This thread receives a traffic video stream containing multiple vehicles of interest as the input frames. The thread has to handle online signals from the camera, video sequences, or series of images. In the case of corruption upon reading, the thread stands by and waits for the following input.

2) *Thread 2 - Preprocessor*: This thread compresses and formats the frames from memory into the batch and pushes them into the memory to further process information with the predefined ROI and MOI settings. Based on the prerequisite of the detector in the GPU, the thread converts the frames into the precisely formatted tensor, compresses them into a batch based on the time-stamp, and sends them to the GPU. The input tensor of the batch is used for fast detection and reduces the impact of the bottleneck. As a result, the time for detection is speedy, and the detector stands by and waits for the input tensor if the input is a consecutive single frame.

3) *GPU*: The GPU loads the vehicle detection model and is ready for the CPU's input at the initial stage. The detector in the GPU reads the input tensor from memory, performs batch recognition of vehicles within view efficiently, and masks each result with the consistent time-stamp lists for the subsequent tracking step. The outcome of the detector is pushed into the memory.

4) *Thread 3 - Tracker*: This thread runs a batch of vehicle detection results from the GPU to match the existing trajectory or create a new moving vehicle, which has a state of "in" or "out" based on the moving object state (described in Section III-A). The thread controls the lifetime of a moving vehicle.

5) *Thread 4 - Counter*: This thread executes the scenario-based movement matching scheme for tracking vehicles (tracklets) in a temporally sequential manner. It uses tracklets predicted to have exited the Obz, which are matched using the MOI by the trajectory movement matching and movement of interest region trajectory matching.

6) *Thread 5 - Writer*: This thread uses the counter result to create the estimation flow and send the outcome to the database. Additionally, the thread takes responsibility to temporarily store the results in short-term memory and transmit statistics via the connection to the server. If corruption in the transmission occurs, the thread will resend the information.

This proposed solution design is experimentally demonstrated to be robust and effective for various scenarios, including diverse outside environments and complex traffic conditions.

IV. EXPERIMENTS

This section presents the implementation and the optimizing process, and the effect on performance. We also compare other methods with our framework.

A. Datasets

In the AI City 2021 dataset [22], there are 31 video clips captured from 20 single camera views with numerous weather (including rain and snow) and lighting conditions

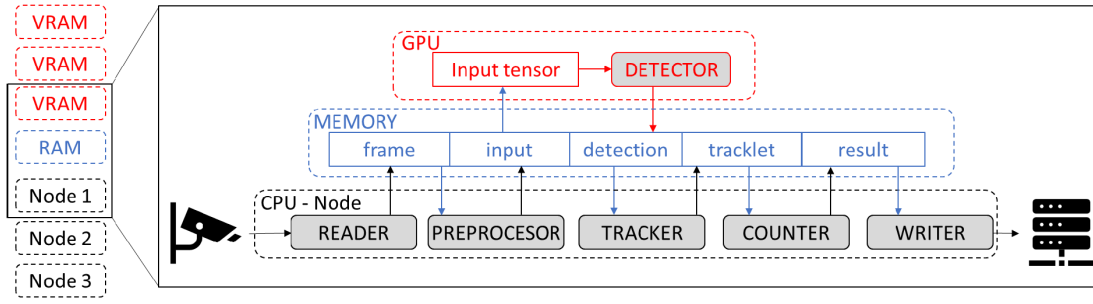


Fig. 7. The implementation of the main pipeline of the framework of the scenario-based movement matching method: we implement an object detection model on a GPU with a tensor format input from the temporary memory. The detector occupies the Video Memory (VRAM). Moreover, we deploy five threads of continuous processing on the CPU and initialize all necessary components of the solution, including a reader, preprocess, tracker, counter, and writer. All of the components are contained in one node in the CPU. The memory storing the tensor and the results from the components is the random access memory (RAM) of the devices.

and several types of road infrastructures (e.g., intersections, highways). According to the AI City 2021 dataset, vehicles are detected and classified into three classes: cars, trucks, and others. We only consider cars and trucks. This is partly because the “others” class includes objects closely following trucks, such as a trailer. The frame rate of the video ranges from 10 to 15 frames per second (FPS). We also selected the UA-DETRAC [23] dataset in training and testing experiments. This dataset consists of 10 hours of videos captured at a framerate 25 frames per second for 24 different locations in Beijing and Tianjin, in China. The six main vehicle types are sedans, sport utility vehicles, vans, taxis, buses, and trucks. Additionally, we also consider a dataset featuring a majority of motorcycle road scenarios. The dataset, which was captured from static pole-mounted traffic surveillance cameras on Vo Van Kiet road [24] in Ho Chi Minh City, Vietnam, was used to assess the accuracy of the counting function using the TSS dataset. Cars, buses, trucks, and motorbikes are labeled in this dataset. The frame rate of each video is 25 FPS. We also used the Human In Events Track 1 [25] dataset for the flow estimation. From the dataset, we selected videos in the folders numbered 3, 10, 11, 12, 14, 17 and 18. Then we added the observation zone for them and manually estimated the flow of each pedestrian on each video.

All datasets are similar to the traffic surveillance video perspective and are suitable for this article’s research topics. The sequence of pictures in each folder in the training set represents a video.

B. Evaluation Metric

The accuracy score is the same as the evaluation score $S1_{effectiveness}$ in the fifth edition of the AI City Challenge [22]. We compute the $S1_{effectiveness}$ score as the weighted average of normalized weighted root mean square error (nwRMSE) scores over all test set films, movements, and vehicle classes. The nwRMSE score is the weighted root-mean-squared error (wRMSE) between the anticipated and actual cumulative vehicle counts, normalized by the actual number of vehicles of that kind in the set of movements. The proportional weights are determined by the number of vehicles of a certain class in a movement. If the wRMSE score is more than the actual number of vehicles, the nwRMSE score

is 0; otherwise, it is $1 - \frac{wRMSE}{vehicle\ count}$. To further lessen the impact of mistakes on early segments, the wRMSE score gives more weight to more recent data by incrementally weighting each record. Each film is divided into k segments to prevent jitters caused by labeling disparities. We assess the movement counts from the beginning of the film to the conclusion of each segment. Minor count errors in early buckets caused by counting before or after the segment breakpoint will lessen as the last segment approaches.

$$wRMSE = \sqrt{\sum_{i=1}^k w_i (\hat{x}_i - x_i)^2} \quad (25)$$

where $w_i = \frac{i}{\sum_{j=1}^k j} = \frac{2i}{k(k+1)}$.

The speed performance is measured by determining the average time of running whole images. The unit is frames per second (FPS). The four datasets have various scenarios for the various kinds of streets with many types of vehicles. We show that the proposed method is scalable and flexible for TFE.

C. Model Training

We train a small model with each camera to obtain a higher accuracy in detection and implement the model for a specific camera. For this goal, we use the YOLOv4 [6], YOLOv5 [7], and PP-YOLO [38]: YOLOv4-Tiny, YOLOv5s, and PP-YOLO_MobileNetV3_small models. Since there are different categories of vehicles in each dataset, we train four versions of each lightweight model corresponding to four datasets with various classes.

We have the strategy to group the cameras from the vehicle they detect. As shown in Fig. 8, we cluster all 35 cameras into 8 clusters. The variables here are the width and height of the bounding box in the first 1000 frames and the ratio of the bounding box’s area over the frame’s area. From 8 clusters, we can train the miniature version of the object detection model based on the dataset that they provide. The accuracy result of the approach of the training method for detector modules is shown in Table X.

The training process is relatively slow and unsuitable for an edge device, so it is performed on a lab workstation. Once trained, the model needs to be migrated and implemented

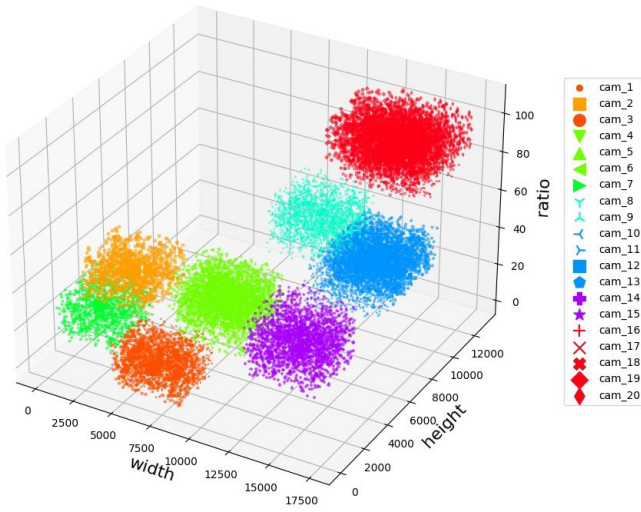


Fig. 8. Bounding box cluster to group the camera in AI City dataset. We use three dimensions for the cluster, including the width and height of the bounding box and the ratio of the area between the bounding box and frame. There are 8 clusters for the 35 cameras.

on the Jetson devices. We convert each model to TensorRT-FP16 for high-performance inference, providing low latency and keeping high throughput. However, the issues are that different models have their own type of structure, and NVIDIA updates the new version TensorRT 8 library on the Jetson devices. To address this problem, at first, with the YOLOv5s, we already have the TensorRT version with the old library, version 7. We migrate the model based on the changelog to TensorRT 8 and make sure the accuracy and the speed performance remain high. Second, for YOLOv4-Tiny and PP-YOLO_MobileNetV3_small, we change the defined network in the YOLOv5 model to the same design in their source code, and then update the weight parameter from the training process. The speed gain and high accuracy are maintained by using TensorRT as shown in Table VI.

D. Migration and Implementation

The section describes the strategy of implementing the algorithm model trained on the lab workstation on the Jetson devices for testing and tuning and deeper implementation of the traffic flow estimation algorithm.

The whole system has been implemented and tested on an NVIDIA Quadro RTX 8000 with 48 GB of memory and an Intel i9-9900X 3.50 GHz processor. To test the capability of the proposed method on an edge device, we deploy and test the algorithm on a Jetson TX2, Jetson Xavier NX, and Jetson AGX Xavier. The hardware information of edge devices is shown in Table III. The hyperparameters we test and optimize for the Jetson devices in the experiment are shown in Table IV. By testing all these hyperparameters, we find the exemplary parameters for each device, and the experiments show that the framework is scalable, flexible, and portable to various computational systems.

We set the default values for various parameters (see Table IV) for the AI City dataset. The queue size is 10, the batch size is 1, the input image size is 768, the number of nodes is 1, and the interval frame is 1.

TABLE III
HARDWARE COMPARISON FOR JETSON MODULES

Feature	TX2	NX	AGX
CPU	4-core ARM Cortex-A57 2-core Denver2	6-core ARM Carmel v8.2	8-core ARM Carmel v8.2
GPU	256-core Pascal	384-core Volta	512-core Volta
Memory	8 GB 128-bit LPDDR4, 58.3 GB/s	8 GB 128-bit LPDDR4 51.2GB/s	16 GB 256-bit LPDDR4 137 GB/s
Tensor cores	-	48	64

TABLE IV
THE HYPERPARAMETER TYPES USED IN THE EXPERIMENT

Type	Value	Experiment
Detector	YOLOv5s, YOLOv4-Tiny, PP-YOLO_MobileNetV3_s	Accuracy, Speed
Tracker	SORT, DeepSORT, FairMOT	Accuracy, Speed
Edge device	TX2, NX, AGX	Accuracy, Speed
Interval multi-frame	1, 2, 3, 4, 5	Accuracy, Speed
Batch size	1, 4, 8	Speed
Image size	352, 384, 416, 640, 768	Accuracy, Speed
Queue size	5, 10, 15	Speed
Number of nodes	1, 2, 3	Speed
Precision	Non-TensorRT, FP16-TensorRT	Accuracy, Speed
Hardware	TX2, NX, AGX	Accuracy, Speed

TABLE V
ACCURACY TEST OF THE INTERVAL FRAMES EXPERIMENT

Interval Frames	Accuracy (%)			Runtime (FPS)		
	AI City	UA-DETRAC	TSS	TX2	NX	AGX
1	90.93	98.60	85.40	12.8	30.4	55.6
2	85.47	97.61	84.55	26.8	49.4	78.6
3	77.29	94.69	80.32	40.8	68.4	101.6
4	65.47	87.11	72.29	54.8	87.4	124.6
5	50.01	75.79	61.44	68.8	106.4	147.6

Note: Batch size = 1, image size = 768, queue size = 10, number of nodes = 1, TensorRT FP16-Precision, YOLOv5s, SORT.

In the beginning, the same network can carry out different resolution detection tasks; therefore, we inspect the model under the condition of multiple normalized scales, as shown in Table VI and Fig. 10a. The smaller the input image scale is, the faster the detection speed. We can adjust the hyperparameter to achieve a balance between speed and accuracy. For the experiment case with a resolution of 768, we obtain the highest accuracy compared to other resolutions; however, the speed performance is lower than the real-time requirement. To tackle this problem, we convert the detection model into TensorRT-FP16 (as mentioned in Section III-B.1), which results in an improvement in the time processing despite slightly lessened accuracy. On the AGX, the effect of the model modification is clearly shown by increasing the processing speed by more than a factor of 1.5 times over the original model, while it lowers the accuracy by nearly 2%. In the case of running the TensorRT model on NX and TX2, we get a considerable reduction in time consumption (as shown in Fig. 10b). Therefore, we use the resolution 768 with the TensorRT version of the detection model to get a high-accuracy result with acceptable processing time.

Second, we consider increasing the framework's speed by using the interval multiframe tracking method [2], and monitor the impact of the method on the accuracy. Instead of inputting the detector with adjacent frames, we drop several images before updating the new one. Table V shows the test in the interval frames technique on three datasets in Jetson devices.

TABLE VI
ACCURACY AND SPEED PERFORMANCE OF THE PROPOSED METHOD ON JETSON DEVICES

Detector	Tracker	Image size	Accuracy		Runtime (FPS)					
					AGX		NX		TX2	
			w/o TRT	with TRT	w/o TRT	with TRT	w/o TRT	with TRT	w/o TRT	with TRT
YOLOv5s	SORT	352	68.12	66.12	41.63	70.39	20.90	38.50	15.23	16.22
YOLOv5s	SORT	384	85.21	82.71	35.46	67.47	17.80	36.90	12.97	15.55
YOLOv5s	SORT	416	78.93	76.61	36.25	68.56	18.20	37.50	13.26	15.80
YOLOv5s	SORT	640	87.58	85.01	33.27	61.80	16.70	33.80	12.17	14.24
YOLOv5s	SORT	768	93.68	90.93	30.48	55.58	15.30	30.40	11.15	12.81
YOLOv5s	DeepSORT	352	69.48	66.78	27.76	46.93	13.93	25.67	10.15	10.81
YOLOv5s	DeepSORT	384	86.92	83.54	23.64	44.98	11.87	24.60	8.65	10.36
YOLOv5s	DeepSORT	416	80.51	77.38	24.17	45.71	12.13	25.00	8.84	10.53
YOLOv5s	DeepSORT	640	89.33	85.86	22.18	41.20	11.13	22.53	8.11	9.49
YOLOv5s	DeepSORT	768	95.55	91.84	20.32	41.69	10.20	22.80	7.43	9.61
YOLOv5s	FairMOT	352	70.18	67.45	26.37	44.58	12.40	22.84	9.03	9.62
YOLOv5s	FairMOT	384	87.78	84.37	22.46	42.73	10.56	21.89	7.69	9.22
YOLOv5s	FairMOT	416	81.31	78.15	22.96	43.42	10.80	22.25	7.87	9.37
YOLOv5s	FairMOT	640	90.23	86.72	21.07	39.14	9.91	20.05	7.22	8.45
YOLOv5s	FairMOT	768	96.51	92.76	19.30	39.60	9.08	20.29	6.61	8.55
YOLOv4-Tiny	SORT	352	55.68	54.05	47.88	80.95	24.04	44.28	17.51	18.65
YOLOv4-Tiny	SORT	384	69.66	67.61	40.78	77.59	20.47	42.44	14.91	17.88
YOLOv4-Tiny	SORT	416	69.50	68.50	41.69	78.85	20.93	43.13	15.25	18.17
YOLOv4-Tiny	SORT	640	71.59	69.49	38.26	71.07	19.21	38.87	13.99	16.38
YOLOv4-Tiny	SORT	768	76.58	74.33	35.05	63.92	17.60	34.96	12.82	14.73
YOLOv4-Tiny	DeepSORT	352	56.80	54.59	30.53	51.62	15.33	28.23	11.17	11.90
YOLOv4-Tiny	DeepSORT	384	71.05	68.29	26.00	49.48	13.05	27.06	9.51	11.40
YOLOv4-Tiny	DeepSORT	416	70.90	69.30	26.59	50.28	13.35	27.50	9.72	11.59
YOLOv4-Tiny	DeepSORT	640	73.02	70.19	24.40	45.32	12.25	24.79	8.92	10.44
YOLOv4-Tiny	DeepSORT	768	78.11	75.07	22.35	45.86	11.22	25.08	8.17	10.57
YOLOv4-Tiny	FairMOT	352	57.37	55.14	29.00	49.04	13.64	25.13	9.94	10.59
YOLOv4-Tiny	FairMOT	384	71.76	68.97	24.70	47.00	11.62	24.08	8.46	10.15
YOLOv4-Tiny	FairMOT	416	71.61	69.99	25.26	47.77	11.88	24.48	8.65	10.31
YOLOv4-Tiny	FairMOT	640	73.75	70.89	23.18	43.05	10.90	22.06	7.94	9.29
YOLOv4-Tiny	FairMOT	768	78.89	75.82	21.23	43.56	9.99	22.32	7.28	9.40
PP-YOLO_MobileNetV3_s	SORT	352	64.41	62.52	44.40	75.06	22.29	41.06	16.24	17.30
PP-YOLO_MobileNetV3_s	SORT	384	80.57	78.20	37.81	71.94	18.98	39.35	13.83	16.58
PP-YOLO_MobileNetV3_s	SORT	416	80.91	79.22	38.66	73.11	19.41	39.99	14.14	16.85
PP-YOLO_MobileNetV3_s	SORT	640	82.81	80.38	35.47	65.90	17.81	36.04	12.97	15.19
PP-YOLO_MobileNetV3_s	SORT	768	88.57	85.97	32.50	59.27	16.32	32.42	11.89	13.66
PP-YOLO_MobileNetV3_s	DeepSORT	352	65.70	63.14	29.60	50.04	14.86	27.37	10.82	11.53
PP-YOLO_MobileNetV3_s	DeepSORT	384	82.18	78.98	25.21	47.96	12.65	26.23	9.22	11.05
PP-YOLO_MobileNetV3_s	DeepSORT	416	82.10	79.01	25.77	48.74	12.94	26.66	9.43	11.23
PP-YOLO_MobileNetV3_s	DeepSORT	640	84.46	81.18	23.65	43.93	11.87	24.03	8.65	10.12
PP-YOLO_MobileNetV3_s	DeepSORT	768	90.35	86.83	21.67	44.45	10.88	24.31	7.92	10.24
PP-YOLO_MobileNetV3_s	FairMOT	352	66.35	63.77	28.71	48.54	13.22	24.36	9.63	10.26
PP-YOLO_MobileNetV3_s	FairMOT	384	83.00	79.77	24.45	46.52	11.26	23.35	8.21	9.84
PP-YOLO_MobileNetV3_s	FairMOT	416	82.92	79.80	25.00	47.28	11.52	23.73	8.39	10.00
PP-YOLO_MobileNetV3_s	FairMOT	640	85.31	81.99	22.94	42.62	10.57	21.39	7.70	9.01
PP-YOLO_MobileNetV3_s	FairMOT	768	91.25	87.70	21.02	43.12	9.68	21.64	7.05	9.12

Note: experiment on a Jetson TX2, a Jetson AGX Xavier, and a Jetson Xavier NX. Batch size = 1, queue size=10, number of nodes = 1, interval frame = 1.

As can be seen, the higher the interval frames, the faster the method runs, and the worse the accuracy becomes. The accuracy decreases by 5% on the AI City dataset when the value of the interval frame is 2. The reason is that the frame rate of the testing video is low (from 10 to 15 frames per second), which leads the IOU between two continuous bounding boxes to decrease and become mismatched, even if these bounding boxes belong to one vehicle. However, for the UA-DETRAC, TSS, Human In Events (whose frame rate is 25), the reductions of results in terms of accuracy are acceptable, with only a 1% loss in accuracy when the number of interval frames is 2, because a higher frame rate keeps the motion smooth and the details crisp. Therefore, we can use a value of 1 for the interval frame with the low frame rate stream and a value of 2 for the high frame rate condition.

Third, with the high processor and memory in GPU, it can operate on more than one image at a time, but when the transferring of data cannot keep up with the processing speed

of the graphics card, a bottleneck occurs. To deal with this issue, we raise the number of images (batch of images) per transfer to Detector and Tracker to increase the system's speed. This approach does not impact the accuracy of the TFE. Fig. 9a describes the testing batch size. The value of 1 is equal to the sequence running, and values of 4 and 8 are equal to the batch of images. For the edge devices, the AGX, which can run with a batchsize of 8, shows more promise than the NX and TX2, which can only reach a batchsize of 4; the hardware in the AGX has more physical memory than others and does not need to extend its memory for implementation. The higher memory, the more effective the implementation is. Therefore, we can set the batchsize to 8 for the AGX and to 4 for the NX and TX2.

Fourth, regarding the ability of the processor and memory, we consider implementing multithreading and memory queue management to boost the speed performance of devices while maintaining high accuracy, as shown in Fig. 7. For more details

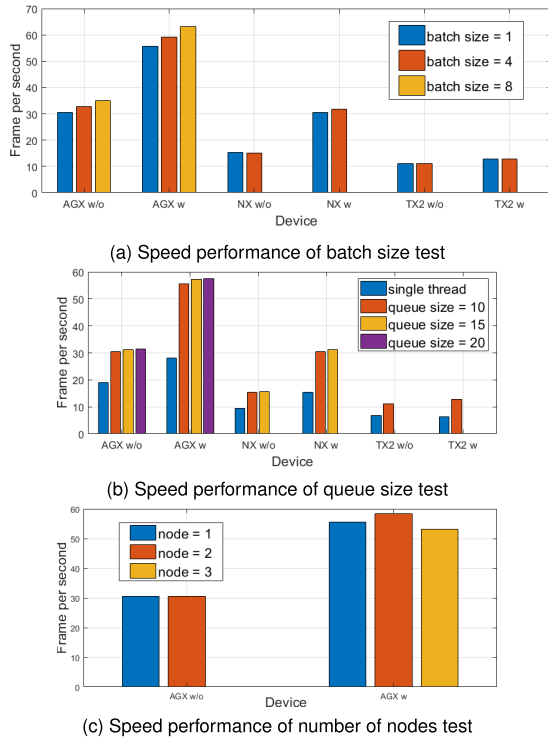


Fig. 9. Speed test of batch size, queue size, and number of nodes on a Jetson TX2, a Jetson AGX Xavier, and a Jetson Xavier NX. The default hyperparameters are batch size = 1, image size = 768, queue size = 10, number of nodes = 1, interval frame = 1, YOLOv5s, SORT, with TensorRT or without TensorRT.

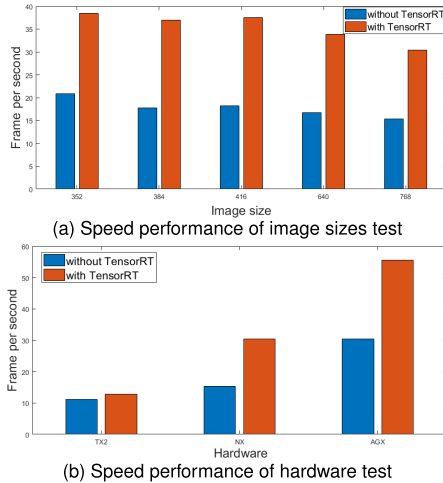


Fig. 10. Speed test of image size and different hardware on a Jetson device. The default hyperparameters are batch size = 1, image size = 768, queue size = 10, number of nodes = 1, interval frame = 1, YOLOv5s, SORT, with TensorRT or without TensorRT.

of the implementation, each node contains five processing threads, including the Reader, Preprocessor, Tracker, Counter, and Writer. Moreover, the size of the queue is the memory that stores the data in each thread. Based on the capability of the device's processor and computational resources, we examined the node we can create and the memory space given for each node. The single thread column shows the speed performance of the system without the multithread deployment. Among the three edge devices we tested (see Fig. 9b), only the AGX can

reach a queue size of 20 while the NX and TX2 acquire queue sizes of 15 and 10, respectively. As can be seen, the more nodes the devices can handle, the faster the system is. The speed performance of multithreading increases by about 10% compared to a single thread. Moreover, the time reduction slightly grows while the queue size rises. Therefore, we can set the queue size as 20 for the AGX, 15 for the NX, and 10 for the TX2. Additionally, we can optimize multithreading by increasing the number of nodes. However, the TX2 and NX cannot handle more than one node. Therefore, we only tested the number of nodes on the AGX (see Fig. 9c). When running the experiment with FP16-Precision, the model could handle three nodes. Nevertheless, The processing time is slower than when using two nodes. Therefore, we can set 2 nodes for the AGX.

E. Evaluation Result

In this section, we compare our method with the state-of-the-art methods, as shown in Table VII. The first column lists the name of each method. Then, the next round of columns shows each technique's components. The two last two columns show the accuracy and speed of each method. For the AI City Dataset, if we use the SORT tracker, the accuracy is marginally lower than the other methods, which use DeepSORT or Tracktor++. In the case of DeepSORT, both the accuracy and speed performance are higher than the others. However, the speed performance is substantially higher than the others. For the UA-DETRAC Dataset, to compare with [2], we changed the default hyperparameters for this experiment, which are 3 for the interval frame, and 352 pixels for the image size. We use both SORT and DeepSORT for comparison. As is shown, the accuracy and speed of our method are higher than those for TFDS [2]. In the instance of the Human In Events Dataset, we configure the identical hyperparameter for this experiment as the UA-DETRAC Dataset. As demonstrated, our method's accuracy and speed are preferable to those of previous methods. In addition to examining and adjusting hyperparameters and modules experimentally, we also participated in Track 1 of the 5th AI City Challenge [22] in the CVPR Workshop to test the efficiency against other frameworks. In this competition, it is necessary to strike a balance between the accuracy and speed performance by designing and optimizing computationally efficient algorithms running on Jetson devices. We placed in the top 3 with the YOLOv5s detector and SORT algorithm in the framework [21].

There are still some challenges in practical applications, and they will need an update in the near future. Weather conditions are the first and most effective system. Adverse conditions, such as heavy rain (as shown in Fig. 11a), following rain, and snow (heavy snow), would make it difficult for the system to identify the correct item. The second issue is posed by the diversity of real-world traffic scenarios (e.g., traffic jams, occlusion) (as shown in Fig. 11b). A bigger vehicle could cover the appearance of a smaller one while stopping at an intersection, which leads to failure of tracking the small vehicle and determining its direction of movement.

TABLE VII
COMPARISON OF RESULT OF THE FRAMEWORK WITH OTHER METHODS

Method	Component in method				Accuracy	Speed (FPS)
	Detector	Tracker	Counter	Multithread		
AI City Dataset - Jetson Xavier NX						
DTC-2020 [19]	Faster R-CNN	DeepSORT	Shape-Based Movement Assignment	-	94.6	2.39
Countor [15]	Faster R-CNN	Tracktor++	Check Line crossing	-	94.6	1.12
Zero-VIRUS [18]	Mask R-CNN	JDE	Route Identification	v	90.4	1.07
DTC-2021 [39]	PP-YOLO	DeepSORT	Shape-based Trajectory Similarity	-	93.4	22.3
Tiny-PIRATE [20]	YOLOv4	DeepSORT	Path-specific Vehicle	v	94.0	20.2
RTM [21]	YOLOv5	SORT	Region-and-Trajectory Movement Matching	v	92.9	20.1
Our	YOLOv5	SORT	Scenario-based Movement Matching	v	93.7	30.4 (+8.1)
Our	YOLOv5	DeepSORT	Scenario-based Movement Matching	v	95.6 (+1.0)	22.8 (+0.5)
UA-DETRAC Dataset - Jetson TX2						
TFDS [2]	YOLOv3	DeepSORT	Virtual line	x	92	37.9
Our	YOLOv5	SORT	Scenario-based Movement Matching	v	94.7 (+2.7)	40.8 (+2.9)
Our	YOLOv5	DeepSORT	Scenario-based Movement Matching	v	96.5 (+4.5)	38.2 (+0.3)
Human In Events Dataset - Jetson Xavier NX						
DTC-2021 [39]	PP-YOLO	DeepSORT	Shape-based Trajectory Similarity	-	89.7	23.4
Tiny-PIRATE [20]	YOLOv4	DeepSORT	Path-specific Vehicle	v	90.3	21.2
RTM [21]	YOLOv5	SORT	Region-and-Trajectory Movement Matching	v	89.2	21.1
Our	YOLOv5	SORT	Scenario-based Movement Matching	v	89.9	31.9 (+8.5)
Our	YOLOv5	DeepSORT	Scenario-based Movement Matching	v	91.7 (+1.5)	23.9 (+0.5)



Fig. 11. The visualization of examples of failures. (a) The heavy rain results in a water drop on the lens and blur view of the camera. (b) The green bus covers the appearance of a small car.

F. Ablation Study

In this section, we show the ablation studies of our method in the Tracker and Counter. In Table VIII, we study the impact of the data association scheme on performance by changing the matching metrics in the method. Specifically, we test the performance of the framework on the Jetson Xavier NX. Each matching metric shows the dilemma in speed and accuracy. The IOU matching has the highest speed performance but the lowest accuracy because it can not handle the missing object in tracking. The Centroid gets the second rank in three of them because of its ability to control the missing detected object. However, high occlusion and changes in the viewpoint could lead to reductions in the accuracy of tracking. The issues could be handled by using Feature matching, even though it obtains the lowest speed performance of the three approaches. Nevertheless, suppose we use only implement Feature matching. In that case, the framework could result in matching the vehicle over a long-distance or in the wrong direction of the current trajectory, which can be treated by using IOU and Centroid matching. In conclusion, we need all three matching metrics to help the tracker obtain high accuracy. The efficiency of our method for only tracker modules is shown in Table XI.

To show the effectiveness of our method in Counter, we experimented with the default setting hyperparameters, and the comparison is the same as in Table VII. However, we keep the same values in the Detector, Tracker, and Multithread

TABLE VIII
ABLATION OF DATA ASSOCIATION

IOU Matching	Centroid Matching	Feature Matching	Accuracy	Speed (FPS)
v			65.3	42.3
	v		77.8	35.6
		v	88.1	25.7
v	v	v	95.6	22.8

Note: Experiment on a Jetson Xavier NX, batch size = 1, queue size=10, number of nodes = 1, interval frame = 1, YOLOv5s, DeepSORT.
'v' stands for whether to use the method.

TABLE IX
ABLATION OF MOVEMENT MATCHING

Method	Accuracy	Speed (FPS)
AI City Dataset - Jetson Xavier NX		
Shape-Based Movement Assignment [19]	89.3	11.3
Check Line Crossing [15]	80.4	16.2
Route Identification [18]	92.7	9.4
Shape-based Trajectory Similarity [39]	93.0	21.3
Path-specific Vehicle [20]	94.4	17.2
Region-and-Trajectory Movement Matching [21]	93.9	18.2
Our-RT	95.3 (+0.8)	21.0
Our-IRT	94.6 (+0.1)	30.2 (+8.9)
Our-All	95.6 (+1.1)	22.8 (+1.5)
UA-DETRAC Dataset - Jetson TX2		
Virtual line [2]	93.4	35.5
Our-RT	96.2 (+2.8)	34.4
Our-IRT	95.5 (+2.1)	40.3 (+4.8)
Our-All	96.5 (+3.1)	38.2 (+2.7)
Human In Events Dataset - Jetson Xavier NX		
Shape-based Trajectory Similarity [39]	89.3	20.4
Path-specific Vehicle [20]	90.7	16.5
Region-and-Trajectory Movement Matching [21]	90.1	17.5
Our-RT	91.4 (+0.8)	20.2
Our-IRT	90.8 (+0.1)	29.0 (+8.5)
Our-All	91.7 (+1.1)	21.9 (+1.4)

Note: experiment batch size = 1, queue size = 10, number of nodes = 1, interval frame = 1, YOLOv5s, DeepSORT.

implementations. We tested the camera feeds using only the Reference Trajectory or Interest Region Trajectory or both. As shown in Table IX, we observe that the method using a reference line could obtain high accuracy. However, this results in a trade-off with the speed performance (Tiny-PIRATE [20], RTM [21]), even though they optimize the

TABLE X
ACCURACY TEST OF DETECTOR MODULES

Detector	mAP
YOLOv5s	95.5
YOLOv4-Tiny	96.4
PP-YOLO_MobileNetV3_s	96.9

TABLE XI
ACCURACY TEST OF TRACKER MODULES

Tracker	MOTA	MOTP	IDF1
SORT	61.20	89.50	70.15
DeepSORT	61.62	91.82	71.66
FairMOT	61.61	92.03	72.10

reference line for lower time-consumption. With the method using a good detector with low-speed performance, with the same miniature version of a detector like this approach uses, the performance reduces significantly (DTC-2020 [19], Zero-VIRUS [18]). Moreover, with implementations of multithreading, the speed performance was boosted considerably. The proposed method only uses the Reference Trajectory (Our-RT) to obtain higher accuracy and lower time performance than for the Interest Region Trajectory (Our-IRT). To harmonize and obtain acceptable results that fulfill the real-time requirement on the Jetson device, we use both of them and obtain the highest accuracy with the acceptable real-time operation.

V. CONCLUSION

In this paper, a scenario-based movement matching method has been proposed. The method has been shown to successfully define the corrected movement direction in a vehicle counting task. The proposed method's performance shows its effectiveness and efficiency in determining a vehicle's route using different camera views. The system achieved third place on the AI City Challenge 2021 Track 1 Dataset A (Track 1: number of vehicles by class at multiple intersections).

We will improve each of the components even further in the future to boost the speed and robustness. For example, we will address typical detection errors, e.g., false positives due to dynamic or reflective backgrounds and a lack of detection of vehicles that are too large or small. Moreover, we will research on better models for the route to adapt to new scenes and weather conditions. In addition to the movement with direction, it is a potential direction to obtain efficient attribute transfer results in a multi-terminal scenario by recreating image characteristics at the decoder used to decode the encoded image. We propose combining our technique with lossless compression for a key-point sequence [40] to encode and transmit vast visual and semantic data between devices, therefore reducing spatial and temporal redundancy. Furthermore, besides the online tracking mentioned in the paper, we can consider offline tracking with post-processing to improve the tracking module and complex scenes, such as TPM [41]. The proposed global tracklet-plane matching associated high-confidence short tracklets with long trajectories. However, TPM is an offline tracking process, where all frames are provided and processed together, and interpolations are

performed to deduce intermediate positions once two object instances across multiple frames are confirmed to have the same identity. By using more information by compressing information from a decode of a key-point sequence, we can focus on processing the data using high-power processing, such as using cloud computing or a large server.

REFERENCES

- [1] B. Tian et al., "Hierarchical and networked vehicle surveillance in ITS: A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 1, pp. 25–48, Jan. 2017.
- [2] C. Chen, B. Liu, S. Wan, P. Qiao, and Q. Pei, "An edge traffic flow detection scheme based on deep learning in an intelligent transportation system," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 3, pp. 1840–1852, Mar. 2021.
- [3] J. Chen, Q. Wang, H. H. Cheng, W. Peng, and W. Xu, "A review of vision-based traffic semantic understanding in ITSs," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 11, pp. 19954–19979, Nov. 2022.
- [4] T. Bouwmans, F. Porikli, B. Hferlin, and A. Vacavant, *Background Modeling and Foreground Detection for Video Surveillance*, 1st ed. New York, NY, USA: Chapman & Hall, 2014.
- [5] W. Liu et al., "SSD: Single shot MultiBox detector," in *Computer Vision—ECCV 2016*. Cham, Switzerland: Springer, 2016, pp. 21–37.
- [6] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," 2020, *arXiv:2004.10934*.
- [7] G. Jocher et al., "ultralytics/yolov5: v5.0—YOLOv5-P6 1280 models, AWS, Supervise.ly and YouTube integrations," Ultralytics, Los Angeles, CA, USA, Tech. Rep. 6.0, Apr. 2021.
- [8] M. Sun, Y. Wang, T. Li, J. Lv, and J. Wu, "Vehicle counting in crowded scenes with multi-channel and multi-task convolutional neural networks," *J. Vis. Commun. Image Represent.*, vol. 49, pp. 412–419, Nov. 2017.
- [9] S. Li, F. Chang, C. Liu, and N. Li, "Vehicle counting and traffic flow parameter estimation for dense traffic scenes," *IET Intell. Transp. Syst.*, vol. 14, no. 12, pp. 1517–1523, Dec. 2020.
- [10] U. Ramer, "An iterative procedure for the polygonal approximation of plane curves," *Comput. Graph. Image Process.*, vol. 1, no. 3, pp. 244–256, Nov. 1972.
- [11] J. Jeon, S. Jung, E. Lee, D. Choi, and H. Myung, "Run your visual-inertial odometry on NVIDIA Jetson: Benchmark tests on a micro aerial vehicle," *IEEE Robot. Autom. Lett.*, vol. 6, no. 3, pp. 5332–5339, Jul. 2021.
- [12] D. N.-N. Tran, L. H. Pham, H.-H. Nguyen, T. H.-P. Tran, H.-J. Jeon, and J. W. Jeon, "Universal detection-based driving assistance using a mono camera with Jetson devices," *IEEE Access*, vol. 10, pp. 59400–59412, 2022.
- [13] H. Yang and S. Qu, "Real-time vehicle detection and counting in complex traffic scenes using background subtraction model with low-rank decomposition," *IET Intell. Transp. Syst.*, vol. 12, no. 1, pp. 75–85, Feb. 2018.
- [14] S. Sooksattra, A. Yoshitaka, T. Kondo, and P. Bunnun, "The density-aware estimation network for vehicle counting in traffic surveillance system," in *Proc. 15th Int. Conf. Signal-Image Technol. Internet-Based Syst. (SITIS)*, Nov. 2019, pp. 231–238.
- [15] A. Ospina and F. Torres, "Countor: Count without bells and whistles," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Seattle, WA, USA, Jun. 2020, pp. 2559–2565.
- [16] A. Fedorov, K. Nikolskaia, S. Ivanov, V. Shepelev, and A. Minbaleev, "Traffic flow estimation with data from a video surveillance camera," *J. Big Data*, vol. 6, no. 1, p. 73, Dec. 2019.
- [17] Z. Dai et al., "Video-based vehicle counting framework," *IEEE Access*, vol. 7, pp. 64460–64470, 2019.
- [18] L. Yu, Q. Feng, Y. Qian, W. Liu, and A. G. Hauptmann, "Zero-VIRUS: Zero-shot vehicle route understanding system for intelligent transportation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Seattle, WA, USA, Jun. 2020, pp. 2534–2543.
- [19] Z. Liu et al., "Robust movement-specific vehicle counting at crowded intersections," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Seattle, WA, USA, Jun. 2020, pp. 2617–2625.
- [20] S. V.-U. Ha, N. M. Chung, T.-C. Nguyen, and H. N. Phan, "Tiny-PIRATE: A tiny model with parallelized intelligence for real-time analysis as a traffic counter," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Nashville, TN, USA, Jun. 2021, pp. 4114–4123.

- [21] D. N.-N. Tran, L. H. Pham, H.-H. Nguyen, T. H.-P. Tran, H.-J. Jeon, and J. W. Jeon, "A region-and-trajectory movement matching for multiple turn-counts at road intersection on edge device," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2021, pp. 4082–4089.
- [22] M. Naphade et al., "The 5th AI city challenge," 2021, *arXiv:2104.12233*.
- [23] L. Wen et al., "UA-DETRAC: A new benchmark and protocol for multi-object detection and tracking," *Comput. Vis. Image Understand.*, vol. 193, Apr. 2020, Art. no. 102907.
- [24] S. V.-U. Ha, D. N.-N. Tran, T. P. Nguyen, and S. V.-T. Dao, "High variation removal for background subtraction in traffic surveillance systems," *IET Comput. Vis.*, vol. 12, no. 8, pp. 1163–1170, Dec. 2018.
- [25] W. Lin et al., "Human in events: A large-scale benchmark for human-centric video analysis in complex events," 2020, *arXiv:2005.04490*.
- [26] Z. Zhou, W. Pei, X. Li, H. Wang, F. Zheng, and Z. He, "Saliency-associated object tracking," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Montreal, QC, Canada, Oct. 2021, pp. 9846–9855.
- [27] M. Danelljan, G. Bhat, F. S. Khan, and M. Felsberg, "ATOM: Accurate tracking by overlap maximization," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Long Beach, CA, USA, Jun. 2019, pp. 4655–4664.
- [28] J. Fan, H. Song, K. Zhang, K. Yang, and Q. Liu, "Feature alignment and aggregation Siamese networks for fast visual tracking," *IEEE Trans. Circuits Syst. Video Technol.*, vol. 31, no. 4, pp. 1296–1307, Apr. 2021.
- [29] Z. Zhou, N. Fan, K. Yang, H. Wang, and Z. He, "Adaptive ensemble perception tracking," *Neural Netw.*, vol. 142, pp. 316–328, Oct. 2021.
- [30] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Sep. 2016, pp. 3464–3468.
- [31] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," in *Proc. IEEE Int. Conf. Image Process. (ICIP)*, Beijing, China, Sep. 2017, pp. 3645–3649.
- [32] Y. Zhang, C. Wang, X. Wang, W. Zeng, and W. Liu, "FairMOT: On the fairness of detection and re-identification in multiple object tracking," *Int. J. Comput. Vis.*, vol. 129, no. 11, pp. 3069–3087, Nov. 2021.
- [33] Y. Zeng, X. Wu, and J. Cao, "The research and analysis of Hungarian algorithm in the structure index reduction for DAE," in *Proc. 11th Int. Symp. Distrib. Comput. Appl. to Bus., Eng. Sci.*, Oct. 2012, pp. 446–450.
- [34] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Fluids Eng.*, vol. 82, no. 1, pp. 35–45, Mar. 1960.
- [35] D. N.-N. Tran et al., "A robust traffic-aware city-scale multi-camera vehicle tracking of vehicles," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Jun. 2022, pp. 3150–3159.
- [36] Y. Yao, L. Zheng, X. Yang, M. Naphade, and T. Gedeon, "Simulating content consistent vehicle datasets with attribute descent," in *Proc. ECCV*, 2020, pp. 775–791.
- [37] D. P. Huttenlocher, G. A. Klanderman, and W. J. Rucklidge, "Comparing images using the Hausdorff distance," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 15, no. 9, pp. 850–863, Sep. 1993.
- [38] X. Huang et al., "PP-YOLOv2: A practical object detector," 2021, *arXiv:2104.10419*.
- [39] J. Lu et al., "Robust and online vehicle counting at crowded intersections," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, Nashville, TN, USA, Jun. 2021, pp. 3997–4003.
- [40] W. Lin et al., "Key-point sequence lossless compression for intelligent video analysis," *IEEE Multimedia*, vol. 27, no. 3, pp. 12–22, Jul./Sep. 2020.
- [41] J. Peng et al., "TPM: Multiple object tracking with tracklet-plane matching," *Pattern Recognit.*, vol. 107, Nov. 2020, Art. no. 107480.



Duong Nguyen-Ngoc Tran (Graduate Student Member, IEEE) received the B.S. degree in computer science and the M.S. degree in information technology management from International University, Ho Chi Minh City, Vietnam, in 2014 and 2018, respectively. He is currently pursuing the Ph.D. degree in electrical and computer engineering with Sungkyunkwan University, Suwon, South Korea.

His current research interests include computer vision, image processing, and deep learning.



computer vision, image processing, and deep learning.

Long Hoang Pham (Member, IEEE) received the B.S. degree in computer science and the M.S. degree in information technology management from International University, Ho Chi Minh City, Vietnam, in 2013 and 2017, respectively, and the Ph.D. degree in electrical and computer engineering from Sungkyunkwan University, Suwon, South Korea, in 2021.

He is currently doing his post-doctoral research with the Automation Laboratory, Sungkyunkwan University. His current research interests include



Huy-Hung Nguyen received the B.S. degree in computer science and the M.E. degree in information technology management from International University-Vietnam National University, Vietnam, in 2014 and 2017, respectively. He is currently pursuing the Ph.D. degree in electrical and computer engineering with Sungkyunkwan University, Suwon, South Korea.

His research interests include computer vision, image processing, and deep learning.



Jae Wook Jeon (Senior Member, IEEE) received the B.S. and M.S. degrees in electronics engineering from Seoul National University, Seoul, South Korea, in 1984 and 1986, respectively, and the Ph.D. degree in electrical engineering from Purdue University, West Lafayette, IN, USA, in 1990.

From 1990 to 1994, he was a Senior Researcher with Samsung Electronics, Suwon, South Korea. Since 1994, he has been with Sungkyunkwan University, Suwon, where he was first an Assistant Professor with the School of Electrical and Computer

Engineering and is currently a Professor with the School of Information and Communication Engineering. His current research interests include robotics, embedded systems, and factory automation.